COMP3121 Notes

Ronnie Low

$$Questions \tag{1}$$

1) for big theta, if big oh and big omega are different then do we have a big theta?
2) Proof by invareints review
3) greedy GSA and exhcange proof reviews (does it look right lmao)

What is an algorithm?
Algorithms are a set of precisely defined steps are executed based off mechanical steps - there is no intelligence involved, just follow the steps given.

Types of algorithms
1. Sequential deterministic algorithms:
- Only one step can be executed at a time
- No element of randomness

2. Parallel algorithms:
- Multiple steps can be ran at a time
- Probably not covered in 3121

3. Randomised algorithms:
- Utilises random values
- Used in quantum computing
- Probably not covered in 3121

Why prove algorithms?
Sometimes, its not obvious if an algorithm does the things its advertised through common sense:
- Will your algorithm terminate in a timely manner?
- Can you prove that the algorithm won't infinitely loop?
- Is the time complexity as advertised?

Proofs are needed for these cases, most of the time they're the **only** way to know if the algorithm does its job.

Stable matching example
There are *n* hospitals and *n* doctors. Each hospital wants to hire exactly one doctor. Each doctor and hospital submit a list of preferences ranking which hospital they want to go to and which doctor they want.

How do we design an algorithm where there is no hospital doctor pair (*h, d*) that are dissatisfied with their result (stable):

$$\text{for pair } p = (h, d') \ \& \ p' = (h', d)$$

- h prefers d over d'
- h' prefers d' ovr d

This implies that both hospitals and doctors have a preferred swap to make (unstable match)

Stable matching algorithm
While there is a free hospital, pick a free hospital h and offer a job to the highest ranking doctor d on its preference list.

If d doesn't have a job yet, d always accepts the job.
else if d is already in a pair $(h'd)$:
- if h is higher ranked on the preference list for d, a new pair (h, d) is created and the previous pair (h', d) is deleted.
- if h is ranked lower, then the offer is ignored.

How many ways to match hospital-doctors?
$n!$, first doctor has n choices, next doctor has $n - 1$ choices, so on and so forth.
Will a stable match always exist?
Yes, but it is not obvious - we need to use a proof:
note that during the while loop:
- for a doctor, the hospital they go to will always be higher on their ranks.
- a hospital will be paired with doctors of decreasing ranks.

If we assume that the ranking is not stable: $p = (h, d)$ and $p' = (h', d')$ such that h prefers d' and h' prefers d.

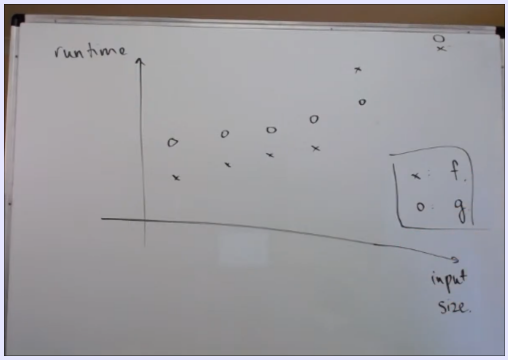
Since h prefers d' to d, then it must have made an offer to d' before offering to d, but since h is paired with d, then d' would need to have rejected h because tehy were already at a hospital they prefer to h. This proves that its impossible to have d' prefer h' if h is ranked higher.

Rates of growth (Recap)
There are statements like:
- *Heap sort is faster than bubble sort.*
- *Linear search is slower than binary search.*

We want our language to be more precise; when is it faster, will it be not the case?

If we were to compare two algorithms, issues may occur!
- Outliers! We can see in the graph above that although g is generally slower, the fifth point has an outlier where f is slower. How do we describe the changes/differences?

- Implementation differences: what if f' is slower than both g and f, but f' has a different implementation?



We mostly perfer to talk in terms of long term behaviour (asymtopics)

For instance, if the size of an input doubles, the function could double, triple, or even quadruple!

Big O Notation

We say that $f(n) = O(g(n))$ if:

There are positive constants C and N such that $0 \leq f(n) \leq C \times g(n)$ for all $n \geq N$.

What does that acutally mean? lets take a look at some examples.

Is $f(n) = O(g(n))$ for:

$f(n) = n + 5$, $g(n) = n^2 - 1$

For this to hold true, there needs to be a value n and C such that every single value of $f(n) \leq g(n)$. **Is this true?**

Well, given a sufficiently large n, yes! Lets prove it:
Lets assign $C = 1$

$$f(n) = n + 5 < g(n) = n^2 - 1 \tag{2}$$
$$0 < n^2 - n - 6 \tag{3}$$
$$0 < (n-3)(n+2) \tag{4}$$
$$\therefore n \text{ is true for } n > 3 \text{ or } n < -2 \tag{5}$$

this implies that for values $C = 1$ and every value $n > 3$, this equation holds true!
In more human terms, we **need** to have a value C that is multiplied with $g(n)$, and there **needs** to exist a value n such that every value greater than n will satisfy

$$0 \leq f(n) \leq C \times g(n).$$

In doing so, we can say that the for $f(n)$ (which is the function of input size vs time) has a time complexity of $O(g(n))$ (keep in mind $g(n)$ will just be an equation that $f(n)$ will never exceed.)

Keep in mind that big O tends to overestimate the complexity (by virute of upper bounding at an asymtopic point).

Is big-oh the be all-end all?

We know that big-oh demonstrates the worst case time complexities. But theres alot of data we're missing out on! What about best case? There are algorithms out there that can run faster than other algos even if they are bounded by the same big-oh time complexity! Think about running bubble sort on a sorted array with the first and last items swapped vs selection sort. This is where big theta and omega comes in.

Big Omega

Big omega is the inverse of big-oh, it yields the asymtopic lower bound of a function. In plain words, $\Omega(f(n))$ is a metric that given a long enough running time, $f(n)$ will always be at least $\Omega(f(n))$. Another way to think about it is that given a time complexity $f(n)$, it will never be less than $\Omega(f(n))$ so plotting it on a graph will show that $\Omega(f(n))$ is bounded by $f(n)$.

Big theta

given that we know $O(f(n))$ and $\Omega(f(n))$, we can combine those two graphs (where the space between big-oh and big omega represents big theta!). This means that the growth rate is the same in both best and worst case senarios.

sum and product property

for all time complexities, we want to know how summing differnet time complexities and the product of them chagnes the total time complexity!

for the sum of time complexities, we take the larger value:

$$O(a) > O(b), O(a) + O(b) = O(a)$$

. As an example, this makes sense!

$$O(n) + O(n^2) = O(n^2)$$

for the product of time complexities, the product of two time complexities will result in the product of the growth rates:

$$O(a) \times O(b) = O(ab)$$

.

Heaps, Frequency tables and Hash tables

To be honest, I don't want to explain this so this will be a challenge left to the reader.

Propositional Logic

- Propositions are true/false statements. We can run operations on them!

- $\neg P$ is a negation of P, i.e. !P.
- $P \vee Q$ is P OR Q, P || Q
- $P \wedge Q$ is P AND Q, P && Q
- $P \Rightarrow Q$ is P implies Q, i.e. if P is true, then Q is true.
- $P \iff Q$ is P if and only if Q which means that for P to true, Q also has to be true, and for Q to be true, then P also has to be true. People have explained iff as $P \Rightarrow Q, Q \Rightarrow P$, but I like to think of it as "they are either both true or they are either both false"

Types of proofs
In 3121, we need to prove algorithms. There are a couple techniques we can employ to do so:

- Proof by induction
- Proof by invarients
- Proof by contradiction

Proof by induction
Proof by induction is pretty fun. Generally, induction is used to prove a sequence of things (that are ordered). If we prove that given any position, the next one holds a certain property and the first item also holds that property, then we can extrapolate that the entire set holds the property. For instance, for an infinitely long set of stairs that are green, if we are given the property that at any green step, the next step will be green.

There exists another kind of induction, strong induction. Normal indcution assumes something at a point: at k, the $k+1$th step is green. However for strong induction, we assume it is true from the base case to k, i.e. $1, 2, ..., k$th steps are all green.

To prove for induction, we have to prove three things:

1) the base case is correct
2) assume it is true for a k (or up to k if we are doing strong induction)
3) prove it is true for the $k+1$th item

An induction proof is not complete without these three properties.

Proof by invarients
Proof by invarients is pretty similar to induciton: We wnat to prove that a property P holds true over an algorithm, and that if P is true at the start, and if we can prove that at any step we can preserve P, then P holds true throughout.

Proof by contradiction
We can prove that something is true by thinking about its negation. If we assume that $\neg P$ is true, and that we somehow prove that this is false, then we can prove that P is true. This feels flimsy but it holds true.

For any statement P, it either has to be true or false. Negating that statement P means that $\neq P$ is the opposite of P. As such, if $\neq P$ is false, then P is true.

Algorithm Reeasoning: Mergesort

- For an algorithm, we need to prove correctness and efficiency
- as such, we need to prove that it gives the right answer and jusitfy the time complexity

Lets take mergesort for example: We would need to prove that not only mergesort "sorts" correctly, but also it runs in $O(nlogn)$ time for a proof to be complete.

Mergesort takes an array $A[l..r]$, and splits it in half every time. If $l = r$, then it simply returns (since there is just one item and theres nothing left to sort).

- Otherwise, you get the midpoint: $(l+r)/2$.
- Apply the algorithm recursively to $A[l, m]$ and $A[m+1, r]$
- merge the subarrays $A[l..m]$ and $A[m+1..r]$, into a sorted subarray $A[l..r]$.

We need a merge operation to successfully merge the two subarrays $A[l..m]$ and $A[m+1..r]$.

- Create a buffer B of size $r - l + 1$.
- fill B from left to right using indices i, j that point to the smallest uncopied elements of the two subarrays
- copy the smaller of $A[i]$ or $A[j]$ until they go out of bounds
- overwrite $A[l..r]$ with B when done.

Mergesort Proof
There are two parts we need to prove correctness for:

- mergesort(A, l, r)
- merge(A, l, m, r)

Lets first prove that merge() successfully creates a sorted array using two sorted subarrays:

- Invarient: At any time the items copied into B so far are the smallest elements of $A[l..r]$ in sorted order.
- Initialisation: This holds since nothing has been copied into B
- Maintanance: Each iteration copies the smallest item between the two arrays, so the invarient holds.
- termination: When all elements have been copied, merge terminates.
- As such, we can prove that given two sorted subarrays, merge() successfully creates a larget sorted array.

What about the mergesort step? Let us use induction.

- Base case: Our array of size 1 is already sorted.
- Inductive assumption: Assume that mergesort correctly sorts all arrays of size up to k

- For an array of size $k+1$, we can represent it as two split ararys of sizes $(k+1)/2$.
- Running mergesort on the two arrays results in two sorted arrays.
- Running merge on both arrays creates a sorted array of size $k+1$.

By induction, the algorithm is correect for all sizes.

Greedy Algorithms

A greedy algorithm is an algorithm that solves a problem by solving it in stages.

At each stage, it selects the best possible choice (that it knows of locally). The difficult part of greedy problems are actually proving them :c

The tradeoff for greedy is that although simple to come up and describe, they don't consider all (or future) possibilities. So proofs are going to be annoying.

Greedy proofs

There are two main ways to prove greedy algos:

- Greedy stays ahead
- exchange argument

For these part of the notes, I'll be making use of a combination of 3121 lecture slides and stanford's guide to greedy algos

Greedy formats

Generally, greedy algorithms are used to solve optimisations (minimise/maximise) based off a set of constraints.

- Maximise attending events without attending overlapping events
- Minimise number of jumps over a pond without falling into the water
- Minimise number of edges chosen while maintaining connectivity.

When we write a greedy proof, we aim for two things:

- Correctness - is it right? does it obey the constraints?
- Optimality - is it optimal? Does it maximise/minimise the quantity you want?

Greedy Stays Ahead

GSA aims to prove that according to some measure, the greedy algorithm is always at least as far as the optimal solution during each iteration of the algorithm. We can use this established fact to prove that greedy does in fact, stay ahead.

The typical GSA format is:

1) Solution definition: Define your variables, your algo will produce some A and gets compared against some A' which is optimal.
2) Cost function: You want to come up with some cost function that can compare both your greedy and optimal solution where you can show the cost of A and A' at every step
3) Prove GSA: we need to prove that the cost of A_i stays ahead or is not lesser than A'_i. This is done inductively
4) Prove optimality: Using the fact that GSA, you can show that greedy is optimal. You can usually do this by assuming that greedy isn't optimal, and proving that greedy stays ahead to derive a conclusion.

Challenge of GSA

- cost functions can be difficult to find
- often better to prove that if greedy stays ahead according to your cost function, then the algorithm is optimal before showing that greedy actually stays ahead.
- another way to think about it is by thinking of greedy as not falling behind

GSA: Cell towers

Let us tackle an example problem: Along the long, straight road from Balladonia (in the West) to Caiguna (in the East), houses are scattered quite sparsely, sometimes with long gaps between two consecutive houses. Telstra must provide mobile phone service to people who live alongside the road, and the range of Telstra's cell tower is 5km.

Task: Design an algorithm for placing the smallest number of cell towers alongside the road, that is sufficient to cover all houses.

Let our greedy solution be to place the cell towers 5km to the right of the leftmost house.

Firstly let us claim that for all $k \geq 1$, the kth tower in the greedy selection is at least as far east as the kth tower in the optimal selection. (here we are assuming that if greedy stays ahead, then our algorithm is optimal).

Let us do this inductively. For the first cell tower, (k = 1), greedy places g_1 5km to the right of the leftmost house. For the optimal solution, it has to cover at least the first house. As such, it can place the cell tower in any interval between $[h_1 - 5..h_1 + 5]$, where h_1 is the first house. We know that $g_1 = h_1 + 5$. The optimal solution cannot place a cell tower further east, since it would not cover h. As such, $g_1 \geq o_1$.

Let us assume that this is true for g_k and o_k:

$$g_k \geq o_k$$

For tower $k+1$, we need to prove that $g_{k+1} \geq o_{k+1}$. Given that we know the leftmost uncovered house h' exists where g_k and o_k doesn't cover h', we know that greedy will place $g_{k+1} - 5 = h$. For the sake of contradition, assume that $o_{k+1} > g_{k+1}$ while covering h. If $o_{k+1} > g_{k+1}$, then $o_{k+1} > h + 5$. But that

would mean taht o_{k+1} is not covering h, so through cotradiction we can prove that $g_{k+1} \geq o_{k+1}$

Exchange Arguments

The exchange argument proves for optimality by showing that you can iteratively transform any optimal solution into the solution provided by the greedy solution without affecting cost and optimality.

1) Define solutions: for a greedy solution G and an optimal solution O, so define variables explicitly
2) Compare solns: Show that if $O \neq G$, then they differ in some way (or at some point). This could be that things are out of order between O and G, or that theres a piece in G that is missing in O, or something else.
3) Exchange pieces: Show that you can transform O by changing some piece in O for some piece of G. this can be order, items, etc, but you need to prove that this exchange did not change the optimality of O and yet have a different solution.
4) Iterate: you need to prove/argue that this exchange has lowered the diferences between O and G and that you can iteratively do this to O to transform it into G without affecting the outcome

More exchange arguments & pitfalls

- The greedy solution could be optimal even if $G \neq O$. There can be multiple optimal solutions
- One thing that helped was thinking of the optimal solution as an "alternate" solution that just happened to be optimal.
- Your solution may need iterative transformations to turn O into G. One modification may make them more similar, but not equal. As such, we need to end our proofs by showing that we can indeed iterate over O so the transformation can happen
- Proof must also explain why exchanging a piece g and o can exist in the first place. Usually this is trivial but you still need to point out that $O \neq G$.

Exchange argument: Tape storage

A list of n files of size L_i have to be stored on a tape. Each file is not equally likely to be needed, and one must start at the beginning of the tape and scan until they find what they need (linear search).

Task: Order the files on the tape such that the expected retrival time is minimised.

Claim 1: For a file f_i that has files infront of it, its retrieval time would be the sum of all files up to it:

$$T_i = (L_1 + ... + L_i)$$

Claim 2: The expected retrieval time of F_i, given that F_i was requested would be the product of p_i with T_i

$$E_i = p_i \times T_i = p_i \times (L_1 + ... + L_i)$$

Claim 3: For the expected retrieval time of the tape, it would be the sum the expected retirieval times of all files:

$$\sum_{i=1}^{n} E_i$$

Aside: Intuitively, we have two variables we need to consider for each file:

- Probability p_i
- Length L_i

An expected outcome of a given set of actions is the average of said actions if it was done infinitely. As such, an intuitive way of weighing the cost of a file f_i is to consider how many times L_i will be passed over to reach some other file compared to how many times f_i is the target file. To justify f_i occupying a certain space, both metrics matter. The higher p_i is, the closer to the front we want it, and the higher L_i is, the further from the front we want it.

We can represent this pull in opposite directions as the ratio of its probability against its size:

$$\frac{p_i}{L_i}$$

Solution: Let us order our files in desending $\frac{p_i}{L_i}$.

Proof: let our greedy solution be G, and some optimal solution be O. G_i and O_i represents the ith file in ascending order. Let us assume at some point $G \neq O$, and diverges at some i that $G_i \neq O_i$ (since if $G = O$, then greedy would be optimal.)

Let us prove by contradiction. Assume that O is minimised, but at some O_i and O_{i+1} that

$$\frac{P_i}{L_i} < \frac{P_{i+1}}{L_{i+1}}$$

Before the swap, the expected retrieval time (E) is

$$\begin{aligned} E = {} & L_1 p_1 + ... \\ & + (L_1 + ... + L_i) P_i \\ & + (L_1 + ... + L_i + L_{i+1}) P_{i+1} \\ & + ... + (L_1 + ... + L_n) + P_n \end{aligned}$$

After swapping F_i and F_{i+1}:

$$\begin{aligned} E' = {} & L_1 p_1 + ... \\ & + (L_1 + ... + L_{i+1}) P_{i+1} \\ & + (L_1 + ... + L_{i+1} + L_i) P_i \\ & + ... + (L_1 + ... + L_n) P_n \end{aligned}$$

As such,

$$E - E' = L_i P_{i+1} - L_{i+1} P_i$$

Let us rearrange our assumption that

$$\frac{P_i}{L_i} < \frac{P_{i+1}}{L_{i+1}} \Rightarrow P_i L_{i+1} < P_{i+1} L_i$$

As such,

$$L_i P_{i+1} - L_{i+1} P_i > 0$$

Therefore, $E > E'$. However, our assumption was that

$$\frac{P_i}{L_i} < \frac{P_{i+1}}{L_{i+1}}$$

, which would imply that E isn't optimal. Through proof by contradiction, a larger $\frac{P_i}{L_i}$ should be ordered first to minimise expected retrieval time. We can conduct this swap can be done iteratively to transform an optimal solution into the greedy solution.

MODULE 2 (come back to ts) Graph applications
theory

Graph applications
aside

Graph applications
aside

What is dynamic programming
Dynamic programming is a teechnique where you solve a problem by decomposing it into subproblems that come up multiple times. As such, you would solve each subproblem once and store it somewhere so its a O(1) access, instead of having to recompute that value.

The benefits of DP is that you can consider multiple (or all) options at each stage without recomputing said stage. More often than not, these subproblems are overlapping (occur multiple times).

DP Overview
Parts of a DP algo:

- A definition of the subproblems
- A recurrence relation (how one subproblem leads to smaller subproblems)
- Base cases, which are the trivial subproblems where recurrences aren't needed

DP execution:

- DP problems are iterative in a bottom up sense. You need to compute smaller subproblems (the base cases) before being able to compute larger and larger subproblems using the recurrence.
- Answers to subproblems will be stored in a lookup table (usually an array indexed by the parameters of the subproblem).

DP format:

- We need to solve the subproblem in an order which follows the dependencies (you need to solve smaller subproblems to solve a subproblem)
- The original problem might be a subproblem, or it can be solved by combining several subproblems.
- Time complexity is usually given by the number of subproblems multiplied by the "average" time taken to solve a subproblem

DP correctness
To prove that a DP is correct:

- The recurrence correectly relates subproblems together
- Base cases are correctly solved
- Overall answer is correctly solved